

1. IDENTIFICACAO DO PROJETO

Projeto:	PP_2026_001_FORENSIA
Titulo:	ForensIA - Sistema Forense de Deteccao de Conteudo Gerado por IA
Versao doc:	1.1 (atualizado em 12/06/2026)
Periodo:	05/02/2026 - 05/11/2026
Equipe:	3 Estudantes (E1: Backend Java E2: Motor IA Python E3: Frontend/QA/Docs)
Status:	EM ANDAMENTO - Fase 5 Desenvolvimento (Motor Texto concluido)

2. OBJETIVO

Sistema forense para deteccao automatizada de conteudo gerado por Inteligencia Artificial em textos, audios, imagens e videos. O sistema classifica o conteudo como HUMANO, HIBRIDO ou IA_GERADO com base em probabilidade calculada por modelos de aprendizado de maquina (HuggingFace Transformers) e, na ausencia do modelo, por heurísticas estatísticas.

3. ESCOPO DO PRODUTO

3.1 Escopo Incluído

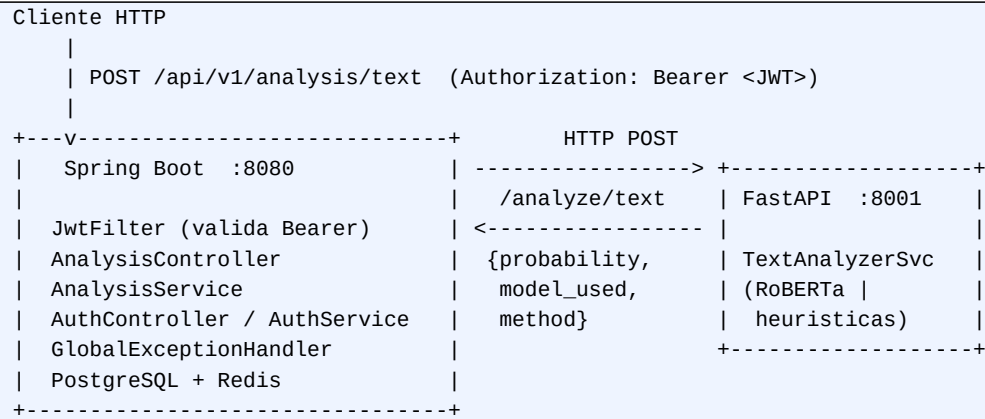
- [+] Cadastro e autenticao de usuarios via JWT (Spring Security + BCrypt)
- [+] Analise de texto: deteccao de conteudo gerado por IA com probabilidade 0.0-1.0
- [+] Classificacao: HUMANO (<30%), HIBRIDO (30-69%), IA_GERADO (>=70%)
- [+] Motor Python com modelo roberta-base-openai-detector + fallback heuristico
- [+] Persistencia de analises no PostgreSQL
- [+] API RESTful documentada (OpenAPI 3.0 / Swagger)
- [+] Containerizacao Docker do motor Python
- [+] Tratamento centralizado de erros e resposta padronizada (ApiResponse<T>)
- [+] (Planejado) Analise de audio: PyTorch + librosa + extracao MFCC
- [+] (Planejado) Analise de imagem: PyTorch + torchvision ResNet-50
- [+] (Planejado) Analise de video: OpenCV + PyTorch + analise de frames

3.2 Escopo Excluido

- [-] Interface Web ou Desktop (frontend definido na fase 5.5)
- [-] Suporte a multiplos idiomas alem de PT-BR e EN
- [-] Retraining do modelo ML (uso de modelo pre-treinado HuggingFace)
- [-] Integracao com sistemas externos de terceiros (exceto HuggingFace Hub)

4. ARQUITETURA TECNICA

4.1 Visao Geral - Dois Microservicos



4.2 Fluxo de Analise de Texto

- 1. Cliente envia POST /api/v1/analysis/text + JWT no header Authorization
 - 2. JwtFilter intercepta -> valida token -> extrai username do SecurityContext
 - 3. AnalysisController recebe TextRequest { Text: '...' }
 - 4. AnalysisService.analyzeText(username, request):
 - a) Monta payload Map { 'text': request.getText() }
 - b) RestTemplate.postForEntity('http://localhost:8001/analyze/text', ...)
 - c) Recebe { probability: 0.87, model_used: '...', method: 'transformer' }
 - d) classify(probability): <0.30=HUMANO | 0.30-0.69=HIBRIDO | >=0.70=IA_GERADO
 - e) Persiste Analysis no PostgreSQL via AnalysisRepository
 - f) Retorna ApiResponse<AnalysisResult>
 - 5. Controller retorna HTTP 200 com JSON padronizado
- [Erro] Se motor Python offline: ResourceAccessException -> GlobalExceptionHandler -> 503

4.3 Stack Tecnologica

Camada	Tecnologia	Versao	Resp.	Porta	Status
Backend API	Spring Boot	3.x	E1	8080	Parcial
Seguranca	Spring Security + JWT	-	E1	8080	Impl.
Banco de dados	PostgreSQL	15	E1	5432	Config.
Cache	Redis	7	E1	6379	Config.
Motor IA	FastAPI	0.115	E2	8001	Concluido
Modelo ML	roberta-base-openai-detector	HuggingF	E2	-	Concluido
Framework ML	PyTorch + HuggingFace Transformers	2.6/4.46	E2	-	Concluido
Containeres	Docker	-	E1/E2	-	Motor OK
CI/CD	GitHub Actions	-	E1	-	Planej.

5. REQUISITOS FUNCIONAIS

Prioridade: Alta = obrigatoria para MVP | Media = importante | Baixa = desejavel

Status: Implementado | Parcial | Planejado | Futuro

ID	Descricao + Criterio de Aceitacao	Prior.	Status
RF01	Cadastro de usuario Sistema registra usuario com username, email e senha (BCrypt).	Alta	Implementado
RF02	Autenticacao via JWT Sistema autentica usuario com username+senha e retorna token	Alta	Implementado
RF03	Analise de texto Sistema recebe texto (min. 5 chars), envia ao motor Python e ret	Alta	Implementado
RF04	Classificacao de conteudo Motor classifica: HUMANO (<30%), HIBRIDO (30-69%), IA_GER	Alta	Implementado
RF05	Persistencia de analises Resultado de cada analise (username, probability, file_type) e ar	Alta	Implementado
RF06	Health check do motor IA GET /health retorna {status: ok} quando motor Python esta opera	Alta	Implementado
RF07	Fallback heuristico Quando modelo ML nao disponivel, motor usa heuristicas estatis	Alta	Implementado
RF08	Historico de analises Usuario autenticado consulta suas analises anteriores paginadas	Media	Planejado
RF09	Analise de audio Sistema aceita arquivo de audio, extrai MFCC via librosa e class	Alta	Planejado
RF10	Analise de imagem Sistema aceita imagem (JPG/PNG), processa com ResNet-50 via	Alta	Planejado
RF11	Análise de vídeo Sistema extrai frames com OpenCV, analisa cada frame com mo	Alta	Planejado
RF12	Interface do usuario Frontend Web ou Desktop permite upload de conteudo e exibe re	Alta	Planejado

6. REQUISITOS NAO-FUNCIONAIS

ID	Categoria	Descricao	Metrica / Criterio
RNF01	Desempenho	Resposta da analise de texto em tempo aceitavel	< 3s para textos ate 512 tokens (modo transformer)
RNF02	Disponibilidade	Sistema deve estar operacional na maior parte do tempo	Uptime >= 99.5% em producao
RNF03	Seguranca	Comunicacao criptografada e autenticacao robusta	HTTPS/TLS 1.3; JWT HS256; BCrypt cost factor >= 10
RNF04	Conformidade	Adequacao a LGPD e GDPR para dados pessoais	DPIA realizado; dados de texto nao persistidos alem do n
RNF05	Escalabilidade	Sistema suporta aumento de carga sem refatoracao	Docker Compose; horizontal scaling via replicas
RNF06	Portabilidade	Sistema executavel em qualquer ambiente com Docker	docker-compose up sobe todo o stack
RNF07	Manutencao	Codigo legivel, testado e documentado	Cobertura de testes >= 70% (JUnit 5 + pytest)
RNF08	Precisao ML	Modelo de deteccao com acuracia aceitavel	F1-score >= 0.80 no conjunto de teste
RNF09	Tolerancia	Falha do motor Python nao derruba o backend Java	GlobalExceptionHandler retorna HTTP 503 graciosamente
RNF10	Auditoria	Todas as analises sao rastreadas por usuario	Tabela analyses com username + timestamp imutaveis

7. REGRAS DE SISTEMA

ID	Regra	Componente	Tipo
RS01	Senha deve conter maiuscula, minuscula e digito	RegisterRequest / LoginsRequest	Validacao
RS02	Username e Email devem ser unicos no banco	AuthService / UserRepository	Integridade
RS03	JWT valido e obrigatorio em /api/v1/**	JwtFilter / SecurityConfig	Autenticacao
RS04	Texto enviado para analise deve ter >= 5 caracteres	FastAPI TextRequest validator	Validacao
RS05	probability >= 0.70 gera campo warning na resposta	AnalysisService.analyzeText()	Negocio
RS06	Motor Python indisponivel retorna HTTP 503	GlobalExceptionHandler	Resiliencia
RS07	Modelo transformer trunca texto em 512 tokens	TextAnalyzerService._load_model()	ML
RS08	Fallback ativado automaticamente se modelo ML falhar	TextAnalyzerService._load_model()	ML
RS09	Role padrao de novo usuario e USER	AuthService.register()	Negocio
RS10	Toda analise persiste username + probability + file_type	AnalysisService + AnalysisRepository	Auditoria

8. MODELO DE DADOS

8.1 Diagrama Entidade-Relacionamento (descritivo)

+-----+ users +-----+	+-----+ analyses +-----+
PK id BIGSERIAL	PK id BIGSERIAL
username -----+-----+> username VARCHAR	
email_user 1 N probability FLOAT	
password_user	file_type VARCHAR
role	role VARCHAR
created_at	created_at TS
+-----+	+-----+

Cardinalidade: 1 usuario -> N analises (referencia por username, sem FK formal)

8.2 Schema PostgreSQL - Tabela users

Coluna	Tipo	Restricoes	Descricao
id	BIGSERIAL	PRIMARY KEY	Identificador unico autoincrement
username	VARCHAR(50)	NOT NULL UNIQUE	Nome de usuario - unico no sistema
email_user	VARCHAR(100)	NOT NULL UNIQUE	Email - unico no sistema
password_user	VARCHAR(255)	NOT NULL	Senha hasheada com BCrypt
role	VARCHAR(20)	NOT NULL DEFAULT 'USER'	Perfil: USER ou ADMIN
created_at	TIMESTAMP	DEFAULT NOW()	Data/hora de criacao

8.3 Schema PostgreSQL - Tabela analyses

Coluna	Tipo	Restricoes	Descricao
id	BIGSERIAL	PRIMARY KEY	Identificador unico autoincrement
username	VARCHAR(50)	NOT NULL	Usuario que realizou a analise
probability	DOUBLE PRECISION	NOT NULL	Probabilidade 0.0 a 1.0 retornada pelo motor
file_type	VARCHAR(20)	NOT NULL DEFAULT 'TEXT'	Tipo: TEXT AUDIO IMAGE VIDEO
role	VARCHAR(20)		Role do usuario no momento da analise
created_at	TIMESTAMP	DEFAULT NOW()	Data/hora da analise

8.4 Modelo JPA - Entidades Java

```

@Entity @Table(name='users')
class User {
    @Id @GeneratedValue Long id;
    String username;        // @Column(unique=true) corrigido: sem unique no Analysis
    String email_user;
    String password_user;   // BCrypt encoded
    String role;            // @Enumerated REMOVIDO - campo String simples
}

@Entity @Table(name='analyses')
class Analysis {
    @Id @GeneratedValue Long id;
    String username;
    Double probability;
    String file_Type;       // 'TEXT' | 'AUDIO' | 'IMAGE' | 'VIDEO'
    String role;
    // created_at gerenciado por @PrePersist ou DEFAULT NOW()
}

```

9. CASOS DE USO

Atores: Usuario (qualquer pessoa), Sistema (componente automatico), Motor IA (FastAPI)

UC01 - Registrar Usuario

Ator:	Usuario (nao autenticado)
Pre-condicao:	Sistema operacional; usuario nao possui conta
Fluxo Principal:	
1. Usuario envia POST /auth/register com { Username, Email_user, Password_user }	
2. AuthController recebe e valida os campos (Bean Validation)	
3. AuthService verifica se username ja existe via UserRepository.existsByUsername()	
4. AuthService verifica se email ja existe via UserRepository.existsByEmail_user()	
5. AuthService codifica senha com BCryptPasswordEncoder	
6. AuthService salva User com role='USER' no banco	
7. Sistema retorna HTTP 200 { sucess: true, data: { message: 'Usuario registrado' } }	
Fluxo Alternativo / Excecao:	
2a. Campo invalido (senha sem maiuscula/digito) -> 400 Bad Request com detalhes	
3a. Username duplicado -> 200 { sucess: false, data: 'Nome de usuario ja esta em uso' }	
4a. Email duplicado -> 200 { sucess: false, data: 'Email ja esta em uso' }	
Pos-condicao:	Novo usuario criado e disponivel para autenticacao

UC02 - Autenticar Usuario (Login)

Ator:	Usuario (cadastrado)
Pre-condicao:	Usuario possui conta ativa no sistema
Fluxo Principal:	
1. Usuario envia POST /auth/login com { Username, Password_user }	
2. AuthController recebe e valida campos	
3. AuthService chama AuthenticationManager.authenticate(UsernamePasswordAuthToken)	
4. Spring Security carrega UserDetails via UserDetailsServiceImpl	
5. BCryptPasswordEncoder compara senha informada com hash no banco	
6. JwtUtil.generateToken(username) gera token JWT assinado (HS256)	
7. Sistema retorna HTTP 200 { token: 'eyJ...', type: 'Bearer', username: '...' }	
Fluxo Alternativo / Excecao:	
5a. Senha incorreta -> BadCredentialsException -> 200 { sucess: false, 'Credenciais invalidas' }	
2a. Campo ausente -> 400 Bad Request	
Pos-condicao:	Token JWT valido emitido; usuario pode acessar endpoints protegidos

UC03 - Analisar Texto

Ator:	Usuario (autenticado com JWT)
Pre-condicao:	Usuario possui JWT valido obtido via UC02
Fluxo Principal:	
1. Usuario envia POST /api/v1/analysis/text com Authorization: Bearer <token>	
2. JwtFilter extrai e valida token; popula SecurityContextHolder	
3. AnalysisController recebe TextRequest { Text: '...' }	
4. AnalysisService.analyzeText(username, request) chamado	
5. AnalysisService faz HTTP POST a http://localhost:8001/analyze/text	
6. Motor Python analisa texto e retorna { probability, model_used, method }	
7. AnalysisService classifica: <0.30=HUMANO 0.30-0.69=HIBRIDO >=0.70=IA_GERADO	
8. Analysis persistida no PostgreSQL (username, probability, file_type='TEXT')	
9. Sistema retorna HTTP 200 com { probability, classification, warning?, created_at }	
Fluxo Alternativo / Excecao:	
1a. JWT invalido/expirado -> JwtFilter -> 401 Unauthorized	
5a. Motor Python offline -> ResourceAccessException -> GlobalExceptionHandler -> 503	
3a. Texto < 5 chars -> FastAPI validator -> 422 Unprocessable Entity	

6a. Motor usa fallback heuristico se modelo ML indisponivel (method='statistical')	
Pos-condicao:	Analise registrada no banco; usuario recebe classificacao e probabilidade
UC04 - Verificar Saude do Motor IA	
Ator:	Sistema (backend Java) ou Administrador
Pre-condicao:	Motor Python iniciado
Fluxo Principal:	
1. Sistema envia GET http://localhost:8001/health	
2. FastAPI retorna HTTP 200 { status: 'ok', service: 'ForensIA Engine' }	
Fluxo Alternativo / Excecao:	
2a. Motor offline -> Connection refused -> Java trata como 503 em UC03	
Pos-condicao:	Status de disponibilidade do motor confirmado

10.1 Sequencia: Registro de Usuario (UC01)

10.2 Sequencia: Login e Obtencao de JWT (UC02)

10.3 Sequencia: Analise de Texto (UC03)

10.4 Sequencia: Motor Python Indisponivel (Cenario de Erro)

ForensIA v1.1 | 12/06/2026 | Pagina 8

			<--ResourceAccessException--
			-->
		<-----	handleAiUnavailable()
<--HTTP 503 {sucess:false, 'Motor de IA indisponivel'}-----			

11. COMPONENTES IMPLEMENTADOS (Jun/2026)

11.1 Motor Python - CONCLUIDO (40/40h)

Microservico FastAPI independente em python-engine/. TextAnalyzerService singleton: modelo carregado uma vez na inicializacao.

Modo Transformer: pipeline HuggingFace openai-community/roberta-base-openai-detector, truncation=True, max_length=512. Label 'Fake'=IA, 'Real'=humano.

Modo Heuristico (fallback): 5 features ponderadas calculadas sobre o texto.

Feature	Peso	Indicador de IA	Formula/Limiar
low_burstiness	30%	Variacao baixa no tamanho das frases	variance/mean_len < 8.0
many_transitions	25%	Uso de conectivos formais (furthermore, portanto...)	min(1.0, count_transitions/3.0)
high_ttr	20%	Type-Token Ratio alto = vocabulario diverso	len(unique_words)/len(words) > 0.65
long_sentences	15%	Frases longas e completas	avg_words_per_sentence > 18
no_personal_pronouns	10%	Escrita impessoal, ausencia de eu/nos/my/our	1.0 se nenhum encontrado

11.2 Backend Java - Auth + Analysis (28/40h)

Arquivo	Pacote	Responsabilidade
UserDetailsServiceImpl	service	Implementa UserDetailsService do Spring Security; carrega User do BD
AuthService	service	Registro (BCrypt) + Login (JWT); verifica unicidade de username/email
AnalysisService	service	Orquestra HTTP ao motor Python; classifica probability; persiste Analysis
AuthController	controller	POST /auth/register e POST /auth/login
AnalysisController	controller	POST /api/v1/analysis/text; extrai username do SecurityContext
GlobalExceptionHandler	exception	Trata ValidationException, ResourceAccessException, BadCredentials, etc.

11.3 Correcoes Aplicadas - java-fixes/ (9 bugs)

Arquivo	Problema Original	Correcao Aplicada
SecurityConfig.java	Path /auth/** (typo na rota de auth)	/auth/**
JwtFilter.java	username == null (logica invertida, linha 49)	username != null
RegisterRequest.java	Regex [?=.*\d] - classe de char incorreta	(?=.*\d) lookahead
LoginsRequest.java	Mesmo regex invalido [?=.*\d]	(?=.*\d) lookahead
Analysis.java	unique=true em campo username da tabela	Removido - nao era chave unica
Analysis.java	@Enumerated em campo String	Removido - campo ja e String
User.java	@Enumerated em campo String role	Removido - campo ja e String
User.java	import javax.swing.* (import errado)	Removido completamente
RestTemplate.java	Classe vazia conflitando com bean Spring	Deletado; adicionado RestTemplateConfig.java

12. STATUS DAS ATIVIDADES - CRONOGRAMA

Atualizacao: 12/06/2026 | Motor Texto: CONCLUIDO | Motor Audio: ATRASADO (1 dia)

SEQ	Atividade	h Est.	h Real.	Inicio	Termino	Resp.	Status
1	Kick-off do Projeto	8	8	05/02	12/02	E1,2,3	FEITO
2	Identificacao de Stakeholders	8	8	10/02	17/02	E1	FEITO
3	Definicao de Escopo	16	16	19/02	05/03	E1	FEITO
4	Plano de Riscos	8	8	26/02	05/03	E1	FEITO
5	Plano de Comunicacao	4	4	26/02	03/03	E1	FEITO
6	Elicitacao de Requisitos	24	24	05/03	19/03	E1,E3	FEITO
7	Documentacao SRS - Marco 1	24	24	12/03	26/03	E3	FEITO
8	Regras de Sistema	16	16	19/03	29/03	E3	FEITO
9	Modelagem de Dados	16	7	02/04	12/04	E1	PARCIAL
10	Design da API REST	16	5	09/04	19/04	E1	PARCIAL
11	Prototipo Interface - Marco 2	16	2	09/04	23/04	E3	PARCIAL
12	Motor Deteccao Texto	40	40	30/04	12/06	E2	FEITO
13	Motor Deteccao Audio	40	-	21/05	11/06	E2	ATRASADO
14	Motor Deteccao Imagem	40	-	11/06	02/07	E2	PENDENTE
15	Motor Deteccao Video	40	-	02/07	23/07	E2	PENDENTE
16	API RESTful Spring Boot	40	28	21/05	02/07	E1	PARCIAL
17	Interface Usuario - Marco 3	40	-	02/07	30/07	E3	PENDENTE
18	Testes Unitarios	24	-	20/08	03/09	E1,E2	PENDENTE
19	Testes de Integracao	20	-	27/08	06/09	E1,E3	PENDENTE
20	Testes de Sistema	20	-	03/09	13/09	E3	PENDENTE
21	Testes do Modelo ML	16	-	03/09	13/09	E2	PENDENTE
22	Testes de Desempenho	12	-	10/09	17/09	E1,E3	PENDENTE
23	Testes de Seguranca - Marco 4	12	-	10/09	17/09	E3	PENDENTE
24	Configuracao de Ambiente	16	-	24/09	04/10	E1	PENDENTE
25	Deploy em Producao - Marco 5	16	-	01/10	08/10	E1	PENDENTE
26	Treinamento de Usuarios	8	-	08/10	13/10	E1,E3	PENDENTE
27	Manual do Usuario	16	-	15/10	29/10	E3	PENDENTE
28	Documentacao Tecnica Final	16	-	22/10	01/11	E1,2,3	PENDENTE
29	Licoes Aprendidas - Marco 6	8	-	01/11	06/11	E1,2,3	PENDENTE
TOTAL		580		190	33% concluido (190/580h)		

13. REFERENCIA DE API

13.1 Java Backend - http://localhost:8080

Metodo	Endpoint	Auth	Body / Descricao
POST	/auth/register	Nenhuma	{ Username, Email_user, Password_user } - Registra novo usuario
POST	/auth/login	Nenhuma	{ Username, Password_user } - Retorna JWT Bearer token
POST	/api/v1/analysis/text	JWT	{ Text } - Analisa texto; retorna probability + classification

13.2 Motor Python - http://localhost:8001

Metodo	Endpoint	Auth	Body / Descricao
POST	/analyze/text	Nenhuma	{ text } - Analisa texto; retorna probability, model_used, method
GET	/health	Nenhuma	Retorna { status: 'ok', service: 'ForensIA Engine' }

13.3 Payloads de Requisicao

```
// POST /auth/register
{ "Username": "usuario01", "Email_user": "user@email.com", "Password_user": "Senha123" }

// POST /auth/login
{ "Username": "usuario01", "Password_user": "Senha123" }

// POST /api/v1/analysis/text (Header: Authorization: Bearer eyJ...)
{ "Text": "Furthermore, it is worth noting that the implementation requires..." }

// POST http://localhost:8001/analyze/text (sem autenticacao)
{ "text": "Furthermore, it is worth noting that the implementation requires..." }
```

13.4 Payloads de Resposta

```
// 200 OK - Analise de texto (Java)
{ "sucess": true, "data": {
  "probability": 0.87, "classification": "IA_GERADO",
  "file_Type": "TEXT", "warning": "Alta probabilidade de conteudo gerado por IA",
  "created_at": "2026-06-12T10:30:00" }}

// 200 OK - Motor Python modo transformer
{ "probability": 0.87, "model_used": "openai-community/roberta-base-openai-detector",
  "method": "transformer", "label": "Fake", "confidence": 0.87 }

// 200 OK - Motor Python modo heuristico (fallback)
{ "probability": 0.75, "model_used": "statistical-heuristics", "method": "statistical",
  "features": { "low_burstiness": 1.0, "many_transitions": 0.67,
    "high_ttr": 1.0, "long_sentences": 0.0, "no_personal_pronouns": 1.0 }}

// 503 Service Unavailable - Motor Python offline
{ "sucess": false, "data": "Motor de IA indisponivel: Connection refused" }

// 401 Unauthorized - JWT invalido
{ "sucess": false, "data": "Credenciais invalidas" }
```

13.5 Classificacao de Probabilidade

Faixa	Classificacao	Descricao
probability < 0.30	HUMANO	Alta confianca de escrita humana
0.30 <= probability < 0.70	HIBRIDO	Conteudo misto ou inconclusivo
probability >= 0.70	IA_GERADO	Alta probabilidade de IA; warning incluso

14. REGISTRO DE RISCOS

I = Impacto (1-5) | P = Probabilidade (1-5) | IR = Indice de Risco (IxP) | IR >= 12 = ALTO

ID	Descricao do Risco	Cat.	I	P	IR	Plano de Resposta / Status
R01	Baixa acuracia do modelo em producao	Tecnico	4	3	12	Mitigado: modelo transformer + fallback heuristico implementado
R02	Mudancas nos LLMs invalidam o modelo	Externo	4	4	16	Aberto: retraining periodico planejado; monitoramento continuo
R03	Nao conformidade LGPD/GDPR	Legal	5	2	10	Aberto: DPIA pendente; dados de texto nao persistidos no motor
R04	Atraso na infra de nuvem	Org.	3	3	9	Mitigado: Docker Compose disponivel para ambiente local
R05	Saida de membro-chave da equipe	RH	4	2	8	Mitigado: codigo documentado em java-fixes/ e java-src/ entregues
R06	Escopo nao aprovado pelo patrocinador	Gestao	3	2	6	Aberto: gates por fase; reunioes semanais de validacao
R07	Motor Python offline degrada a API Java	Tecnico	4	3	12	Mitigado: GlobalExceptionHandler captura ResourceAccessExcept
R08	Vazamento de dados submetidos	Seguranca	5	2	10	Aberto: HTTPS/TLS 1.3 planejado; dados em memoria apenas dur

15. ANALISE PERT - DURACOES POR FASE CRITICA

Formula: PERT = (O + 4M + P) / 6 | O=Otimista M=Mais Provavel P=Pessimista

Fase / Atividade Critica	O (sem)	M (sem)	P (sem)	PERT (sem)	Resp.	Status
Motor de Deteccao - Texto	4	8	12	8.0	E2	FEITO
Motor de Deteccao - Audio	4	8	12	8.0	E2	ATRASADO
Motor de Deteccao - Imagem	4	8	14	8.3	E2	PENDENTE
Motor de Deteccao - Video	4	8	14	8.3	E2	PENDENTE
API RESTful Spring Boot	4	6	10	6.3	E1	PARCIAL
Interface do Usuario	2	5	8	5.0	E3	PENDENTE
Testes e QA	1	2	4	2.2	E1/E3	PENDENTE
Implantacao e Deploy	1	2	3	2.0	E1	PENDENTE
DURACAO TOTAL ESTIMADA (PERT)				48.1 sem.		

16. LEGENDA E CONVENCOES

E1 = Estudante 1 - Lider / Backend Java (Spring Boot + PostgreSQL + Redis + JWT)
E2 = Estudante 2 - Motor IA / Python (FastAPI + HuggingFace + PyTorch + librosa + OpenCV)
E3 = Estudante 3 - Interface / Testes / Documentacao

STACK: Backend -> Java 21 + Spring Boot 3.x + PostgreSQL 15 + Redis 7
Motor IA -> Python 3.11 + FastAPI 0.115 + HuggingFace Transformers 4.46 + PyTorch 2.6
Analise de Audio (planejado) -> librosa + MFCC features
Analise de Imagem (planejado) -> torchvision + ResNet-50
Analise de Video (planejado) -> OpenCV + frame sampling
CI/CD -> Docker + GitHub Actions

17. HISTORICO DE VERSOES DO DOCUMENTO

Versao	Data	Autor	Descricao das Alteracoes
1.0	05/02/2026	E1,E2,E3	Versao inicial - Identificacao, cronograma, riscos, PERT
1.1	12/06/2026	E1,E2	Motor Texto concluido (40h); 9 bugs Java corrigidos; Auth+Analysis endpoints implementados; adi

Documento gerado automaticamente em 12/06/2026 18:23 - ForensIA v1.1